Thematic Questions

Problem A — Dense vs. Sparse Matrix-Vector Multiplication

Setup

Computation environment, fixed throughout: Intel Core Ultra 5 225H (14 cores), 32 GiB DDR5, Linux 7.0.1, Python 3.13.12, NumPy 2.4.4, SciPy 1.17.1, scipy-openblas 0.3.31 with all cores enabled. The matrix dimension is fixed at $\mathbf{N = 4000}$; `A` is the output of `init_matrices(N, p)` and `x` is a fixed `float64` vector of length N. Each timing is the minimum of 50 timed `A @ x` runs after 3 warm-up runs. The Python construction loop inside `init_matrices` runs only once and is excluded from the timing, consistent with the stated assumption that the target application multiplies the same matrix.

A-1 — Benchmark and discussion

The benchmark driver `problem_a1.py` sweeps the non-zero density p and times `A_dense @ x` against `A_sparse @ x` (CSR). Selected rows of the resulting `benchmark_results.json` are reproduced below.

p	nnz	dense (ms)	sparse (ms)	speed-up
0.001	16 051	1.60	0.014	× 118
0.010	160 225	1.73	0.131	× 13
0.050	802 034	1.73	0.756	× 2.3
0.100	1 601 646	1.58	1.21	× 1.3
0.150	2 402 221	1.58	1.80	× 0.88
0.200	3 202 938	1.64	2.28	× 0.72

Dense throughput reaches ~ 20 GFLOP/s; CSR throughput reaches ~ 2.7 GFLOP/s for $p \geq 0.05$.

Throughput analysis. Both kernels are bounded by memory bandwidth rather than by floating-point throughput. The dense call dispatches to OpenBLAS `dgemv`, which streams every entry of `A`: the matrix occupies $N^2 \cdot 8$ B $= 128$ MiB, so reading it in 1.6 ms corresponds to approximately **80 GB/s** — close to the practical DDR5 ceiling of this machine (~ 90 GB/s). The CSR (Compressed Sparse Row) format stores only the non-zero entries (one 8-byte value plus one 4-byte column index per non-zero, plus a small row-pointer array); the SciPy implementation is in C but single-threaded. At $p \geq 0.05$ it sustains approximately 16 GB/s, the bandwidth a single core can extract from one DDR5 channel. Both kernels therefore operate at their respective hardware limits, and the dense-vs-CSR gap is primarily a consequence of the dense kernel exploiting all 14 cores while CSR uses one.

Effect of density on performance.

1. Dense execution time is essentially independent of p (≈ 1.6 ms throughout): multiplications by zero entries consume the same memory bandwidth and floating-point cycles as non-zero entries, so zeros are not free under the dense format.
2. CSR execution time grows linearly with the number of non-zeros; doubling p doubles the time (1.21 ms at $p = 0.1 \rightarrow$ 2.28 ms at $p = 0.2$).
3. The crossover lies at approximately $\boldsymbol{p \approx 0.13}$ (15% density). Beyond this point the 4-byte column index per non-zero plus the absence of multi-threading in the CSR kernel outweigh the saving from skipping zero entries.
4. The crossover is sensitive to thread count. Setting `OPENBLAS_NUM_THREADS=1` raises the dense time to 3.9 ms and pushes the crossover beyond $p = 0.2$, so the appropriate format also depends on the BLAS thread configuration.

Memory footprint similarly favours CSR: $\approx 12pN^2$ bytes versus the dense $8N^2$, so CSR is the smaller representation up to roughly $p \approx \frac{2}{3}$.

A-2 — PageRank as an SpMV-bound application

Definition. PageRank assigns to each page i of a directed web graph a numerical importance $x[i]$, defined as the stationary distribution of a **random surfer** who at each step either follows a uniformly random outgoing link (with probability α) or teleports to a uniformly random page (with probability $1 - \alpha$). The teleport term, parameterised by the **damping factor** $\alpha = 0.85$, makes the underlying Markov chain irreducible and aperiodic — so the stationary distribution is unique — and bounds the spectral gap, which controls the convergence rate of the iterative solver. With P the row-stochastic transition matrix of the link graph, the rank vector satisfies the fixed-point equation

$$x = \alpha \cdot P^T x + \frac{1-\alpha}{N} \cdot \mathbf{1}.$$

Reduction to repeated SpMV. The standard solver is the **power iteration**: starting from $x^{(0)} = \mathbf{1}/N$, the right-hand side above is applied repeatedly. Each iteration is one CSR multiplication by the fixed matrix P^T plus an $O(N)$ vector update; the residual shrinks by a factor of α per iteration, so 50–100 iterations reach a tolerance of 10^{-8} to 10^{-10}. Real web graphs have only 10–30 outgoing links per page regardless of N, so at $N = 10^8$ the matrix has $\text{nnz} \approx 3 \cdot 10^9$ (36 GB in CSR) while the dense representation would require 10^{16} entries and is not a viable storage option — the operating point is the very-low-density regime of Section A-1, where CSR is already 100–400 times faster than dense.

Measured cost. The script `problem_a2.py` runs the power iteration above using the SpMV kernel of Section A-1, with `P` taken to be the row-normalised output of `init_matrices`, $N = 4000$, $\alpha = 0.85$, tolerance 10^{-10}, and at most 200 iterations.

p	iters	total SpMV (ms)	per iter (μ s)	SpMV / total
$1 \cdot 10^{-3}$	93	1.33	14	63%
$5 \cdot 10^{-3}$	16	0.79	50	84%
$1 \cdot 10^{-2}$	13	1.77	137	88%
$1 \cdot 10^{-1}$	8	14.36	1795	98%

The SpMV share rises from 63% to 98% as the matrix becomes denser; the vector update and renormalisation are negligible whenever the matrix is non-trivial, so the throughput measured in Section A-1 directly predicts the application throughput. Extrapolating to $N = 10^7$ with average out-degree 20 ($\text{nnz} = 2 \cdot 10^8$), the CSR kernel of Section A-1 yields 1.5 s per SpMV; 80 power iterations therefore cost roughly 2 minutes of wall time — practical, but with all non-SpMV work invisible.

SpMV specialisations for the PageRank matrix

1. **Binary-matrix SpMV (factor out the row-stochastic scaling).** Every non-zero of P has the value $1/d_{\text{out}(i)}$, which depends only on the row, so $P^T = A^T D$ where $A \in \{0,1\}^{N \times N}$ is the binary link adjacency and $D = \text{diag}(1/d_{\text{out}})$. The PageRank step becomes $P^T x = A^T(Dx) = A^T z$ with $z = Dx$ pre-computed in $O(N)$. The SpMV then reads only column indices, no values, so bytes per non-zero drop from 12 to 4 — a 3 × reduction in memory traffic on this memory-bound kernel. The decomposition is unavailable to a generic SpMV because arbitrary matrix entries are not products of a row-only factor and a binary indicator.
2. **Reorder columns by host, then delta-encode indices.** Real web graphs have 80% of links staying within the same host. After permuting page IDs so that pages on the same host are contiguous, the column indices within each row of P^T are tightly clustered — typically within a span of a few thousand — so CSR can store column **deltas** in 16 bits instead of absolute 32-bit indices, shrinking index bytes by 2--4 ×. Combined with item 1 the SpMV reads 2 B per non-zero. The reordering is paid once and reused; the speed-up depends on a property of web graphs, not of sparse matrices in general.
3. **Hub-row chunking for power-law in-degree.** In-degree on a web graph follows a power law: a few pages have 10^5+ in-links while the median has under 10. A row-partitioned parallel CSR SpMV gives catastrophic thread imbalance — the thread owning a hub row works thousands of times longer than its neighbours. A PageRank-tuned kernel splits any row longer than a threshold (chosen at the inflection point of the measured in-degree distribution) into chunks that several threads sum into the same $y[j]$ via partial accumulators. Uniform-padding formats (ELLPACK / Sliced-ELLPACK) cannot do this without wasting enormous storage on the long tail.
4. **Fused SpMV + dangling correction + AXPY.** The full PageRank step is $x \leftarrow \alpha(P^T x) + (\alpha s + 1 - \alpha)/N \cdot \mathbf{1}$ with $s = \sum_{i \in D} x[i]$ over dangling pages D. A textbook pipeline performs this in three separate passes over y (SpMV, scale, add bias). A specialised kernel emits each $y[j]$ in a single streaming pass over CSR — row sum, scale by α, add the pre-computed scalar bias — and updates s for the next iteration on the same pass. This roughly halves the per-iteration y-traffic on the memory-bound kernel.

Problem B — An Accelerator for LLM Inference: Hardware and Software

1. The shape of LLM inference

A large language model is a stack of transformer blocks. To produce text from a prompt, the chip runs two phases that look completely different to the silicon underneath. **Prefill** sends the whole prompt through every layer once; the layer is dominated by a few large matrix multiplies of shape roughly $(L \times d) \times (d \times d)$, where L is the prompt length and d is the hidden width. With L in the hundreds, every byte of weight read from memory feeds hundreds of multiply-accumulate operations, so prefill is **compute-bound**: peak FLOPs decide its speed. **Decode** then generates tokens one at a time; each new token re-uses the same weights with $L = 1$, every layer collapses to a matrix × vector, and the chip does only a handful of operations per byte read. Decode is therefore **memory-bandwidth-bound**: GB/s decide its speed, and FLOPs are mostly idle. A 70-billion-parameter model in 8-bit weights occupies 70 GB; generating **one** token requires reading all of it at least once, which on a 3 TB/s HBM stack costs 23 ms before any arithmetic. The bandwidth-bound regime is the one users feel as latency, so we treat it as the primary design constraint.

2. Architectural characteristics

2.1 A memory system designed around the decode roofline

The dominant cost during decode is moving bytes, not computing on them, so the memory subsystem is the most important block on the chip. Two layers matter.

Off-chip — HBM. HBM (High-Bandwidth Memory) is DRAM stacked vertically right next to the compute die and connected through a wide silicon interposer; relative to the DDR DIMMs in a normal server, it delivers 5–10× the bandwidth at far lower energy per byte, because the wires are short and very numerous. A modern accelerator ships with 4–8 stacks giving a total of 3–5 TB/s and 80–200 GB of capacity. **Both numbers matter:** if the model plus its KV-cache (the keys and values produced for every previous token, kept so that attention does not recompute them) does not fit on-package, weights spill to host memory over PCIe and effective bandwidth drops by an order of magnitude. Capacity, not just bandwidth, is therefore a first-class design parameter.

On-chip — SRAM and operator fusion. HBM bytes are precious, so the chip should re-read as little as possible. The lever is on-chip **SRAM** — a small but very fast memory built into the same silicon as the compute units, with nanosecond latency and bandwidth orders of magnitude above HBM. With enough SRAM (tens of MiB per compute cluster) the compiler can **tile** big tensors into blocks that fit on chip and **fuse** several operators into a single pass over those blocks. The canonical case is attention. The naive implementation forms an $L \times L$ score matrix, writes it to HBM, reads it back, softmaxes it, multiplies by V, and writes again — four trips across the bandwidth wall whose total size scales as L^2. If a tile of queries, keys and values fits in SRAM, the score block, the softmax and the value-product all happen on chip, and only the final output tile is written back. This is FlashAttention, and it is the single largest software-visible reason to spend transistors on SRAM: on long contexts it cuts attention bandwidth by more than $10\times$ and turns attention from a memory-bound bottleneck into a compute-bound block. The same tiling logic fuses the GEMM epilogue (bias, RMSNorm, RoPE rotation, KV-cache append) into the preceding matrix multiply, so the activation tensor is never spilled to HBM between layers. SRAM size therefore directly sets how aggressively the compiler can fuse, and how close decode runs to the HBM roof.

2.2 Matrix engines tuned for low precision

Inside each compute tile, the bulk of the work is dense matrix multiplication. The standard structure is a **systolic array**: a 2-D grid of multiply-accumulate cells through which operands flow in lock-step, so each value read from SRAM feeds many multiplications before being discarded. The array's economic value is set almost entirely by its supported numeric formats. Inference, unlike training, tolerates very low precision: there is no backward pass to amplify rounding error, and post-training calibration can absorb most of the remaining loss. In practice, **FP8** (8-bit float, with separate E4M3 and E5M2 variants chosen per-tensor for activations vs. gradients-of-attention) is now the default for both weights and activations, doubling throughput over FP16 with sub-percent accuracy loss, and **INT4 weight-only** quantisation (weights stored in 4 bits, activations kept in FP16/FP8) halves the weight-bandwidth bill on decode — where weight bandwidth is **the** bottleneck — at the cost of a small dequantise step inside the GEMM epilogue. The matrix engine must therefore expose FP16, BF16, FP8 and INT4 as first-class input types with **FP32 accumulation** (a wide running sum keeps the result accurate even when the inputs are

narrow). Treating low precision as a fallback rather than a primary path is the most common way for a chip to look fast on paper and slow on real workloads.

2.3 Chip-to-chip interconnect

A 70-B model in 8-bit fits on one chip; a 400-B Mixture-of-Experts model does not, and even a 70-B model is usually sharded across several chips to push decode latency below 50 ms/token. The dominant sharding pattern is **tensor parallelism**: every weight matrix is split column-wise across k chips, and an `all-reduce` collective sums the partial activations after each block. During decode the activation being reduced is tiny (one row of width d, a few KB), so what kills performance is **latency per collective**, not bandwidth. The interconnect must therefore offer (i) sub-microsecond latency between chips in the same node, (ii) of order 1 TB/s of bisection bandwidth to keep prefill all-reduces from becoming the bottleneck, and (iii) hardware support for **overlap**: launching the all-reduce of layer ℓ while the matrix engines start layer $\ell + 1$, so communication and compute share the wall clock instead of stacking. Without it, decode latency stops scaling past two or three chips.

3. Software stack

3.1 A tensor compiler with attention-aware fusion

The frontend (PyTorch, JAX) hands over a static computation graph, which a compiler lowers through an **intermediate representation** (a typed, hardware-neutral description of the graph, similar in spirit to LLVM IR but with tensors as the primitive type) such as MLIR, OpenXLA or TVM. The IR's job on this hardware is concrete: choose tile shapes that match the systolic array and fit the SRAM budget, fuse every element-wise op into the GEMM that produces its input, lower quantised types to the right hardware instruction, and emit the asynchronous DMAs that prefetch the next tile while the current one computes. The frontier feature is **attention-aware fusion**: instead of treating attention as a sequence of generic ops, the compiler recognises the pattern and emits a single kernel templated over head size, sequence length and KV-cache layout (paged or contiguous). The hand-written kernel library still exists — FlashAttention, cuBLAS-style GEMMs, MoE routing — but as a **fallback and a correctness oracle**; on a well-engineered compiler stack the auto-generated path closes most of the gap.

3.2 A serving runtime built around the KV-cache

A trained model becomes a useful product only when many users share one accelerator. The runtime is what makes that sharing efficient, and it is structured entirely around the KV-cache, because the KV-cache is the largest and most awkward tensor in the system: it grows linearly with context length, varies per request, and must persist across decode steps. Two ideas dominate.

Paged KV-cache. Allocating one contiguous slab per request leads to internal fragmentation comparable to early operating systems' contiguous allocation: a request that **might** reach 32 K tokens reserves 32 K tokens' worth of GPU memory from the start, even if it stops at 200. The fix, popularised by vLLM, is to allocate the KV-cache in fixed-size **blocks** (say 16 tokens each) and maintain per-request page tables that map logical token positions to physical blocks. Two requests that share a prompt prefix (a system message, a few-shot template) can then share the prefix blocks by reference, paying the prefill cost only once. The hardware needs nothing more than fast gather/scatter with indirect addressing, but the software win is several-fold: realised throughput typically rises 2--4 × at fixed latency.

Continuous batching. A naive scheduler decodes a batch to completion before admitting new requests; short replies then wait for the longest one in the batch, and the chip is half-idle for most of the window. Continuous batching merges new requests into the running decode batch **every step**, so a request that finishes in 50 tokens frees its slot immediately for an arriving prefill, and a long request keeps decoding without interruption. The runtime must therefore co-schedule prefill and decode in the same step (a **chunked prefill** breaks long prompts into pieces that fit alongside ongoing decodes), respect per-request priorities, and hand the matrix engine a different shape every iteration. **Speculative decoding** sits on top: a small draft model proposes several tokens, the main model verifies them in one prefill-shaped pass, and accepted tokens skip a full decode step — a 2–3× speed-up at no quality loss.

3.3 Quantisation toolchain

The FP8 and INT4 paths in Section 2.2 are only useful if a trained BF16 checkpoint can be lowered into them without retraining. An offline **quantisation toolchain** (GPTQ, AWQ, SmoothQuant) uses a small calibration set — a few hundred prompts — to choose per-channel scales that preserve activation shapes, and an outlier-handling step that keeps a fraction of weight columns in higher precision when their dynamic range demands it. Small in code volume, it is the bridge that makes the chip's headline FP8/INT4 throughput reachable on real models.

Appendix 1 — Benchmark programs for Problem A

The two scripts below are the source code referenced in Sections A-1 and A-2.

A.1 `problem_a1.py`

```
"""Problem A-1: benchmark dense vs. CSR sparse matrix-vector multiplication.

Usage: ``python problem_a1.py``  (run inside the project's devenv shell).

The script fixes the matrix dimension ``N`` and sweeps the non-zero density
``p``. For each ``p`` it times ``A @ x`` for both the dense ``np.ndarray``
and the ``scipy.sparse.csr_array`` returned by ``init_matrices``. The result
table is printed to stdout and a machine-readable copy is written to
``benchmark_results.json`` for use by the report.

The target application is assumed to multiply the same matrix by many
different vectors (Hint 4 of the problem statement), so the one-shot
construction cost inside ``init_matrices`` is excluded from the timed region.
"""
from __future__ import annotations

import json
import platform
import time
from dataclasses import dataclass, asdict
from pathlib import Path

import numpy as np
import scipy as sp

from problem_a import init_matrices


N = 4000
DENSITIES = [0.0, 0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.15, 0.2]
N_WARMUP = 3
N_ITERS = 50


@dataclass
class Row:
    p: float
    nnz: int
    dense_min_ms: float
    dense_med_ms: float
    sparse_min_ms: float
    sparse_med_ms: float
    dense_gflops: float
    sparse_gflops: float
    dense_bytes_per_s: float
    sparse_bytes_per_s: float


def _time_matvec(A, x: np.ndarray, n_warmup: int, n_iters: int) -> tuple[float, float]:
    """Return ``(min, median)`` wall-clock seconds for a single ``A @ x`` call."""
    if n_iters < 1:
        raise ValueError("n_iters must be >= 1")
    y = A @ x
    for _ in range(n_warmup):
        y = A @ x
    samples = np.empty(n_iters, dtype=np.float64)
    for i in range(n_iters):
        t0 = time.perf_counter()
        y = A @ x
        t1 = time.perf_counter()
        samples[i] = t1 - t0
    # Touch ``y`` so the runtime cannot dead-code-eliminate the multiplications.
    if not np.isfinite(y).all():
        raise RuntimeError("non-finite output from matvec")
    return float(samples.min()), float(np.median(samples))


def run_benchmark() -> list[Row]:
```

```
    rng = np.random.default_rng(42)
    x = rng.standard_normal(N).astype(np.float64)

    rows: list[Row] = []
    for p in DENSITIES:
        A_dense, A_sparse = init_matrices(N, p)
        assert A_dense.shape == (N, N)
        assert A_sparse.shape == (N, N)

        # Sanity check: dense and CSR must produce the same A @ x.
        assert np.allclose(A_dense @ x, A_sparse @ x), \
            f"dense/sparse result mismatch at p={p}"

        d_min, d_med = _time_matvec(A_dense, x, N_WARMUP, N_ITERS)
        s_min, s_med = _time_matvec(A_sparse, x, N_WARMUP, N_ITERS)

        nnz = int(A_sparse.nnz)
        # Useful FLOPs are 2*nnz (one multiply + one add per non-zero). For
        # dense the kernel processes every entry, so the *executed* FLOPs are
        # 2*N*N regardless of how many entries happen to be zero.
        dense_flops = 2.0 * N * N
        sparse_flops = 2.0 * nnz
        # Memory traffic for streaming SpMV (lower bound):
        #   dense  : N*N*8                (matrix) + 2*N*8         (x, y)
        #   sparse : nnz*(8+4) + (N+1)*4  (data + indices + indptr) + 2*N*8
        dense_bytes = N * N * 8 + 2 * N * 8
        sparse_bytes = nnz * (8 + 4) + (N + 1) * 4 + 2 * N * 8

        rows.append(
            Row(
                p=p,
                nnz=nnz,
                dense_min_ms=d_min * 1e3,
                dense_med_ms=d_med * 1e3,
                sparse_min_ms=s_min * 1e3,
                sparse_med_ms=s_med * 1e3,
                dense_gflops=dense_flops / d_min / 1e9,
                sparse_gflops=sparse_flops / s_min / 1e9 if nnz > 0 else 0.0,
                dense_bytes_per_s=dense_bytes / d_min,
                sparse_bytes_per_s=sparse_bytes / s_min,
            )
        )
        print(
            f"p={p:>6.4f}  nnz={nnz:>9d}  "
            f"dense {d_min*1e3:>7.2f} ms ({rows[-1].dense_gflops:>5.2f} GF/s)  "
            f"sparse {s_min*1e3:>7.2f} ms ({rows[-1].sparse_gflops:>5.2f} GF/s)  "
            f"speedup x{(d_min/s_min):>6.2f}",
            flush=True,
        )
    return rows


def env_info() -> dict:
    return {
        "python": platform.python_version(),
        "platform": platform.platform(),
        "processor": platform.processor(),
        "numpy": np.__version__,
        "scipy": sp.__version__,
        "N": N,
        "n_warmup": N_WARMUP,
        "n_iters": N_ITERS,
    }


def main() -> None:
    info = env_info()
    print("Environment:", json.dumps(info, indent=2))
    rows = run_benchmark()

    out = {
        "env": info,
        "rows": [asdict(r) for r in rows],
```

```
    }
    Path("benchmark_results.json").write_text(json.dumps(out, indent=2))
    print("\nSaved benchmark_results.json")


if __name__ == "__main__":
    main()
```

A.2 problem_a2.py

```
"""Problem A-2: PageRank power iteration as an SpMV-bound application.

Usage: ``python problem_a2.py``  (run inside the project's devenv shell).

PageRank is the stationary distribution of a random surfer on a directed
graph: at each step the surfer follows a uniformly random outgoing link
from the current page (with probability ``alpha``) or teleports to a
uniformly random page (with probability ``1 - alpha``). With ``P`` the
row-stochastic transition matrix of the link graph, the rank vector ``x``
is the unique solution of the fixed-point equation

    x = alpha * P^T @ x + (1 - alpha) / N * 1.

The standard solver is power iteration: starting from ``x = 1 / N``, the
right-hand side is applied repeatedly until convergence. Each step is one
CSR multiplication with the *fixed* sparse matrix ``P^T`` plus an O(N)
vector update, so the script exercises the same SpMV implementation as
problem_a1.py and reports the share of total wall-time spent inside the
SpMV. The random matrix returned by ``init_matrices`` is row-normalised so
that ``P`` is a valid row-stochastic transition matrix.
"""
from __future__ import annotations

import time
from dataclasses import dataclass

import numpy as np
import scipy as sp

from problem_a import init_matrices


N = 4000
DENSITIES = [0.0001, 0.001, 0.005, 0.01, 0.05, 0.1]
ALPHA = 0.85
TOL = 1e-10
MAX_ITER = 200


@dataclass
class PageRankResult:
    n_iters: int
    final_residual: float
    total_time_s: float
    spmv_time_s: float


def row_stochastic_csr(A: sp.sparse.csr_array) -> sp.sparse.csr_array:
    """Return a copy of ``A`` with each non-zero row scaled to sum to 1.

    Rows whose entries are all zero (dangling nodes) are left as zero rows.
    """
    A = A.copy()
    row_sums = np.asarray(A.sum(axis=1)).ravel()
    inv = np.zeros_like(row_sums)
    mask = row_sums > 0
    inv[mask] = 1.0 / row_sums[mask]
    return (sp.sparse.diags_array(inv) @ A).tocsr()


def pagerank(P_T: sp.sparse.csr_array, alpha: float = ALPHA,
             tol: float = TOL, max_iter: int = MAX_ITER) -> PageRankResult:
    """Run PageRank power iteration to convergence (or until ``max_iter``).

    ``P_T`` is the transpose of the row-stochastic transition matrix; we
    take it as input so the per-step kernel is exactly one CSR SpMV.
    Returns timing statistics together with the final L1 residual.
    """
    N_ = P_T.shape[0]
    x = np.full(N_, 1.0 / N_, dtype=np.float64)
    bias = (1.0 - alpha) / N_
```

```
    spmv_time = 0.0
    t_start = time.perf_counter()
    residual = float("inf")
    k = 0
    for k in range(1, max_iter + 1):
        t0 = time.perf_counter()
        y = P_T @ x
        spmv_time += time.perf_counter() - t0

        x_new = alpha * y + bias
        # Renormalise to absorb the rank mass that leaks through dangling rows.
        x_new /= x_new.sum()

        residual = float(np.abs(x_new - x).sum())
        x = x_new
        if residual < tol:
            break
    total_time = time.perf_counter() - t_start
    return PageRankResult(n_iters=k, final_residual=residual,
                          total_time_s=total_time, spmv_time_s=spmv_time)


def main() -> None:
    print(f"PageRank power iteration  N={N}  alpha={ALPHA}  tol={TOL}  "
          f"max_iter={MAX_ITER}")
    header = (f"{'p':>8}  {'nnz':>10}  {'iters':>6}  "
              f"{'spmv (ms)':>10}  {'spmv/iter (us)':>15}  "
              f"{'spmv/total':>10}  {'residual':>12}")
    print(header)
    for p in DENSITIES:
        _, A_sparse = init_matrices(N, p)
        P = row_stochastic_csr(A_sparse)
        P_T = P.T.tocsr()
        res = pagerank(P_T)
        spmv_per_iter_us = res.spmv_time_s / res.n_iters * 1e6
        print(f"{p:>8.4f}  {A_sparse.nnz:>10d}  {res.n_iters:>6d}  "
              f"{res.spmv_time_s*1e3:>10.2f}  {spmv_per_iter_us:>15.1f}  "
              f"{res.spmv_time_s/res.total_time_s:>9.1%}  "
              f"{res.final_residual:>12.2e}")


if __name__ == "__main__":
    main()
```

Appendix 2 — Generative AI Usage Disclosure

```
# Generative AI Usage Disclosure

## Service Name
Claude 4.7 Opus

## How it was used
I utilized Generative AI as my primary tool for solving, drafting, and typesetting the responses
for both Problem A and Problem B. The AI was used end-to-end to generate the solutions, refine the
technical content, and format the final document.
I use AI agents in the following iteration:
1. Give it the original question description, and let it generate the first answer prototype.
2. Review the current answer, confirming the correctness.
3. If the answer is unsatisfying, give it prompt for adjust the answer in the way I want and back
to step 2. Otherwise (the answer looks good to me), stepping to step 4.
4. Stop and submit the answer.

## Prompts Entered

### Problem A Prompts

    "Solve the problems showed in the screenshot."

    "The word is too hard in the report. Use more easier word. And also, many hard concept has not
been described or described in detail. After modifcation, shrink the content to fulfill the 2 A4
page constraint."

    "The content is still too much. The table uses much space"

    "Rewrite it in typst (don't delete the old one)"

    "Looks good. Add more spaces between sections, and make the font a little bit larger."

    "Need you to give two modifications of A-2: 1. Write more details about the PageRank (and why
it is related to the problem). 2. About the optimizations of SpMV: you should specialize to the
PageRank problem, not for generic case. For example, Multi-thread SpMV should not be considered
specialization."

    "No, don't modify the pagerank itself. The specialization is, only modify the algorithm of
SpMV for optimization. And this modification is for pagerank but not generic."

    "Looks good. Content looks OK. But the sentence of this report looks like an answer to my
request. Remember this report is the the report to the 'problems' (in the screenshot before), but
not me. Adjust the sentences and formats"

### Problem B Prompts

    "Solve the problem in this description (Please answer the following within two A4 pages total.
Use Typst for writing. Discuss the architectural characteristics required for an accelerator
designed for LLM inference, as well as the software stack needed to support that architecture. ##
Additional Requirements - Please start from easy to hard. From simple concept and then go deeper -
You should write in a report format, with clear structure. - Do not use hard words - If some hard
concept you have to use, describe them in detail first)"

    "I think you want to include too much information, make too much points. Cut off some points
and focus on the rest, give them more details and go deeper on each of them"
```